

Mini Project #2: Octave-spaced FIR filter banks

Spencer G, Luke W, Rami L

November 3rd, 2025

1.0 Introduction

In this mini-project, we are separating the sound file “Wabash Cannonball” into different octaves by creating and implementing a filter bank for it via discrete-time analysis. Within this filter bank, a variety of low- and high-pass filters are used alongside several downsampling and upsampling operations to generate waveforms for the different octaves.

2.0 Complex sinusoidal response of FIR filters cascaded with downsampling and upsampling

We start by proving the validity of the responses of a variety of FIR filters with unique frequency responses after cascading with downsampling, upsampling, and against other frequency responses. We examine how complex sinusoidal signals behave when passing through digital signal processing systems that combine FIR filters with sampling rate changes. Here, we are analyzing four system configurations: an FIR filter followed by downsampling, upsampling followed by an FIR filter, and finally cascades of both operations.

The goal is to derive expressions for the output signals and determine the effective frequency response of the cascaded system. Understanding these relationships is important for multirate signal processing applications, where we need to predict how signals are modified when sampling rates are changed.

2.1

2.1

$$Y_1[z^n] = h_1[z^n] * X_1[z^n]$$

frequency domain,
use $h(z) = H(e^{j\omega})$

$$Y_1[z^n] = H_1(e^{j\omega}) X_1[z^n]$$

$$Y_1[z^n] = H_1(e^{j\omega}) \cdot e^{j2\omega n}$$

$$X_1[n] = e^{j\omega n}$$

$$X_1[z^n] = e^{j2\omega n}$$

2.2

2.2

$$Y_2[n] = h_2[n] * X_2[n]$$

frequency domain

$$Y_2[n] = H_2(e^{j\omega}) X_2[n]$$

$$\cos^2(\omega) = \frac{1}{2} + \frac{1}{2} \cos(2\omega)$$

$$\cos^2\left(\frac{\pi n}{2}\right) = \frac{1}{2} + \frac{1}{2} \cos(\pi n)$$

$$Y_2\left[\frac{n}{2}\right] = H_2(e^{j\frac{\omega}{2}}) X_2\left[\frac{n}{2}\right]$$

$$Y_2\left[\frac{n}{2}\right] = \frac{1}{2} H_2(e^{j\frac{\omega}{2}}) e^{j\frac{\omega}{2} n} + \frac{1}{2} H_2(e^{j\frac{\omega}{2} + \pi}) e^{j(\frac{\omega}{2} + \pi) n}$$

$$X_2[n] = e^{j\omega n} = (\cos(\omega n) + j\sin(\omega n))$$

$$X_2\left[\frac{n}{2}\right] = e^{j\omega \frac{n}{2}}$$

$$X_2\left[\frac{n}{2}\right] = \begin{cases} X_2\left[\frac{n}{2}\right] & n \text{ is even} \\ 0 & n \text{ is odd} \end{cases} = \cos^2\left(\frac{\pi n}{2}\right) \cos(\omega \frac{n}{2})$$

we have
no samples
at $n = \frac{1}{2}, \frac{3}{2}, \dots$

$$X_2\left[\frac{n}{2}\right] = X_2\left[\frac{n}{2}\right] \left(\frac{1}{2} + \frac{1}{2} \cos(\pi n)\right)$$

$$= \frac{1}{2} X_2\left[\frac{n}{2}\right] + \frac{1}{2} X_2\left[\frac{n}{2} - \pi\right]$$

$$= \frac{1}{2} e^{j\frac{\omega}{2} n} + \frac{1}{2} e^{j(\frac{\omega}{2} + \pi) n}$$

2.4

down sample

$$x[n] = e^{j\omega n}$$

$$y[n] = H_1(e^{j\omega}) e^{j\omega 2n}$$

up sample

$$y[\frac{n}{2}] = H_1(e^{j\omega}) x[\frac{n}{2}] \cdot H_1(e^{j\omega})$$

$$\approx \left[\frac{1}{2} H_1(e^{j\omega}) e^{j\omega n} + \frac{1}{2} H_2(e^{j(\omega-\pi)}) e^{j(\omega-\pi)n} \right] H_1(e^{j\omega})$$

don't bring this down b/c it's an alias.

$$y[n] = H_1(e^{j\omega}) \frac{1}{2} H_2(e^{j\omega}) \cdot x[n]$$

$$y[n] = \frac{1}{2} H_1(e^{j\omega}) \cdot H_2(e^{j\omega}) \cdot e^{j\omega n}$$

2.4

down sample

$$y_1 = H_1(e^{j\omega}) e^{j2\omega n} \rightarrow \text{found in 2:1}$$

down sample

$$y_2 = H_2(e^{j2\omega}) H_1(e^{j\omega}) e^{j4\omega n} \rightarrow \text{found in 2:1}$$

up sample

$$y_3 = \left[\frac{1}{2} H_3(e^{j2\omega}) e^{j2\omega n} \right] H_1(e^{j\omega}) H_2(e^{j2\omega})$$

$$= H_1(e^{j\omega}) H_2(e^{j2\omega-2}) \frac{1}{2} H_3(e^{j2\omega}) e^{j2\omega n}$$

$$y_4 = \left[\frac{1}{2} H_4(e^{j\omega}) e^{j\omega n-2} \right] H_1(e^{j\omega}) H_2(e^{j2\omega}) \frac{1}{2} H_3(e^{j2\omega})$$

$$y[n] = \frac{1}{4} H_1(e^{j\omega}) \cdot H_2(e^{j2\omega}) \cdot H_3(e^{j2\omega}) \cdot H_4(e^{j\omega}) \cdot e^{j\omega n}$$

$$H_{\text{eff}} = \frac{1}{4} H_1(e^{j\omega}) \cdot H_2(e^{j2\omega}) \cdot H_3(e^{j2\omega}) \cdot H_4(e^{j\omega})$$

3.0 Effective frequency response of individual branches.

In this section of the experiment, we are given four fundamental FIR low-pass and high-pass filters to analyze the frequency characteristics of a two-level octave-band filter bank. These filters are arranged in a tree structure with cascaded downsampling and upsampling operations to decompose the input signal into three distinct frequency bands. We start with examining the individual filter responses to understand their basic characteristics. Then, using the theoretical results derived in Section 2.0 for multirate systems, we calculate the effective frequency response for each branch. Our analysis shows how the cascade of filtering and sampling rate conversion operations shapes the overall frequency selectivity of each path through the filter bank.

```
% Code accompanying fall 2025 mini project #2 to apply two level DWT
% (discrete wavelet transform) and inverse
% Originally written by Dan Jacobellis 10/24/2025
% The coefficients below for a dyadic perfect reconstruction filterbank %
are based on the Cohen-Daubechies-Feauveau wavelet (bior6.8) % If the
```

```

wavelet toolbox is installed, you can also use the following code: % [LA,
HA, LS, HS] = wfilters('bior6.8');
% LA = LA(2:end)/sqrt(2); HA(2:end) = HA/sqrt(2);
% LS= sqrt(2)*LS(2:end); HS = sqrt(2)*HS(2:end);
coeffs = [
    0.00134974786501001 0
    0 -0.00269949573002003
    -0.00135360470301001 0
    0 -0.00270720940602003
    -0.0120141966670801 0.0102009221870399
    0.0204018443740798 0.0240283933341602
    0.00843901203981008 -0.0102300708193699
    0.0204601416387398 0.0168780240796202
    0.0351664733065404 -0.0556648607799594
    -0.111329721559919 -0.0703329466130807
    -0.0546333136825205 0.0285444717151497
    -0.0570889434302994 -0.109266627365041
    -0.0665099006248407 0.295463938592917
    0.590927877185834 0.133019801249681
    0.297547906345713 -0.536628801791565
    1.07325760358313 0.595095812691426
    0.584015752240756 0.295463938592917
    0.590927877185834 -1.16803150448151
    0.297547906345713 0.0285444717151497
    -0.0570889434302994 0.595095812691426
    -0.0665099006248407 -0.0556648607799594
    -0.111329721559919 0.133019801249681
    -0.0546333136825205 -0.0102300708193699
    0.0204601416387398 -0.109266627365041
    0.0351664733065404 0.0102009221870399
    0.0204018443740798 -0.0703329466130807
    0.00843901203981008 0
    0 0.0168780240796202
    -0.0120141966670801 0
    0 0.0240283933341602
    -0.00135360470301001 0
    0 -0.00270720940602003
    0.00134974786501001 0
    0 -0.00269949573002003
];
LA = coeffs(:,1); % Lowpass Analysis
HA = coeffs(:,2); % Highpass Analysis
LS = coeffs(:,3); % Lowpass Synthesis
HS = coeffs(:,4); % Highpass Synthesis

% Example audio file built into matlab.

```

1

```

audiodata = load('handel.mat');
x = audiodata.y(1:end-1); fs = audiodata.Fs;

% Analysis (level 1)

```

```

L1 = downsample(conv(x,LA),2);
H1 = downsample(conv(x,HA),2);

% Analysis (level 2)
L1L2 = downsample(conv(L1,LA),2);
L1H2 = downsample(conv(L1,HA),2);

% Synthesis
delay = length(coeffs)-1;

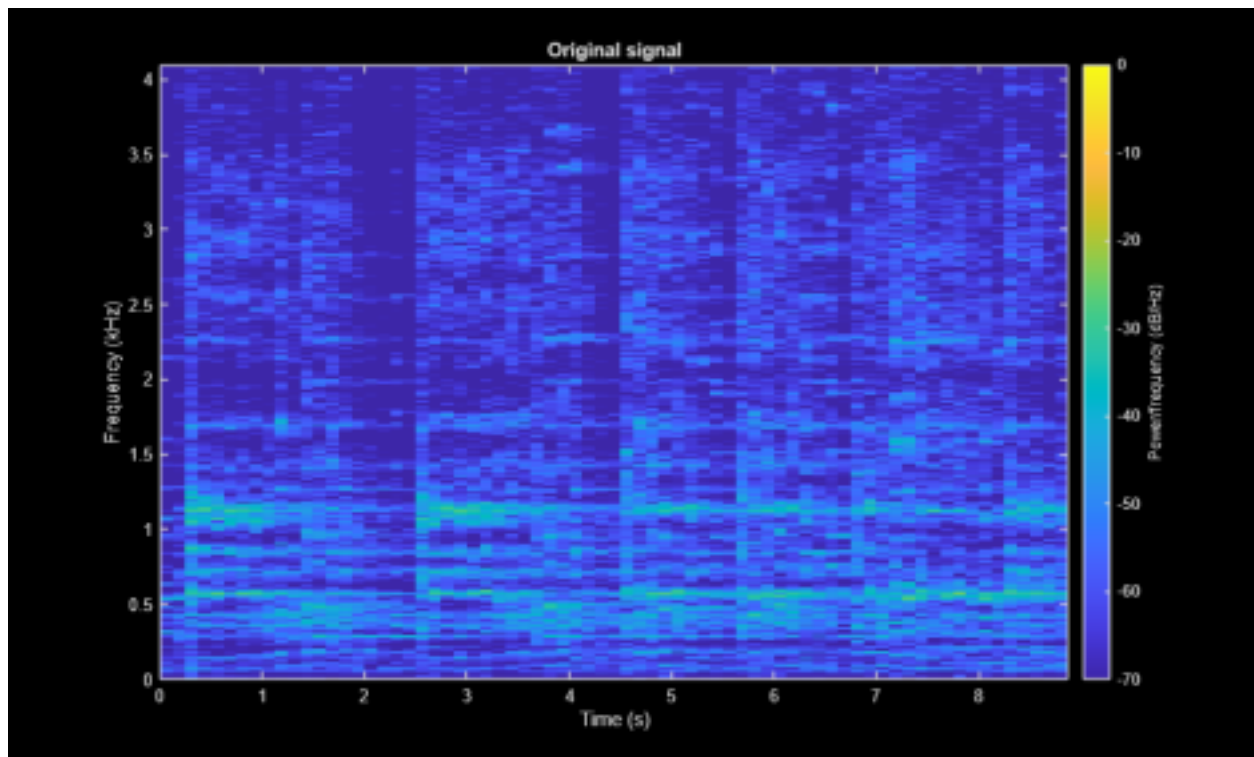
L1rec = conv(upsample(L1L2,2),LS) + conv(upsample(L1H2,2),HS);
L1rec = L1rec(delay:end-delay-1);

xrec = conv(upsample(L1rec,2),LS) + conv(upsample(H1,2),HS);
xrec = xrec(delay:end-delay-1);

% verify that xrec is the same as x (accounting for small numerical error)
assert( max(abs(x-xrec)) < 1e-10 )

% create spectrograms for original signal and components
figure; spectrogram(x,1024,0,1024,'yaxis',fs);
clim([-70,0]); title('Original signal')

```

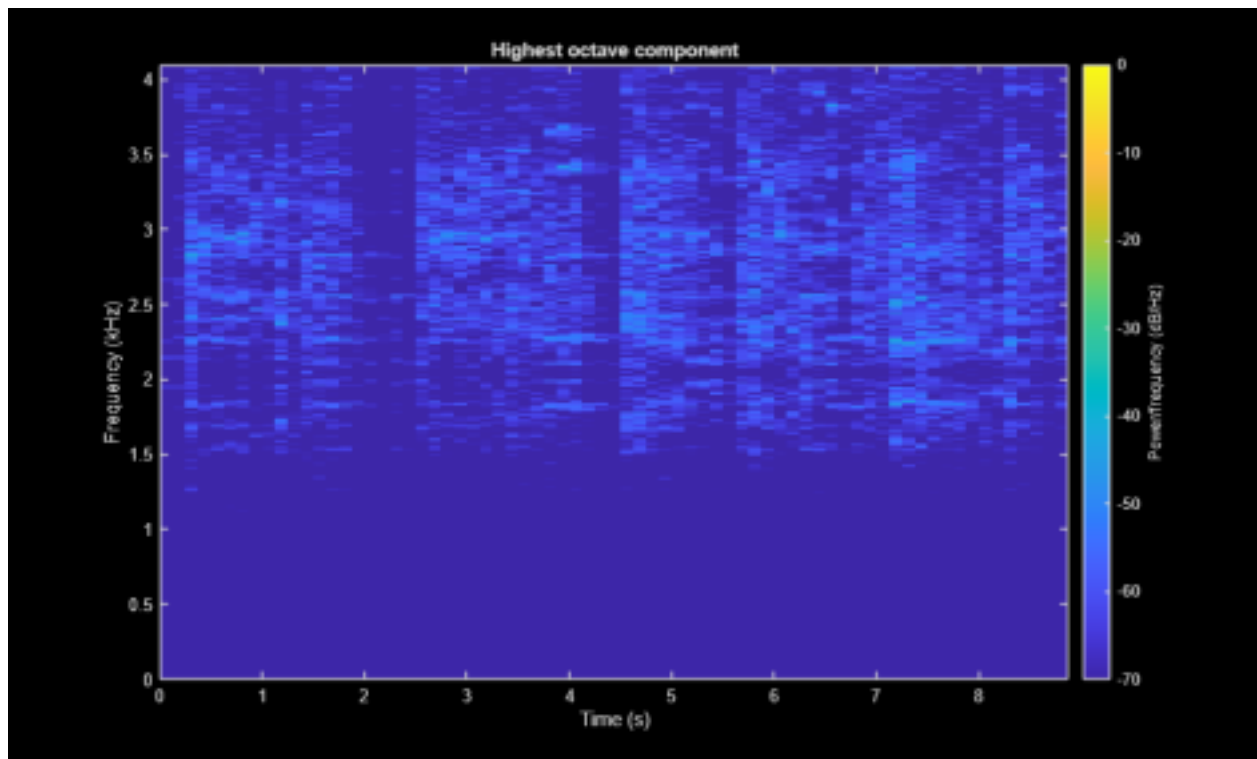


Spectrogram of original signal.

```

highest_octave = conv(upsample(H1,2),HS);
figure; spectrogram(highest_octave,1024,0,1024,'yaxis',fs);
clim([-70,0]); title('Highest octave component');

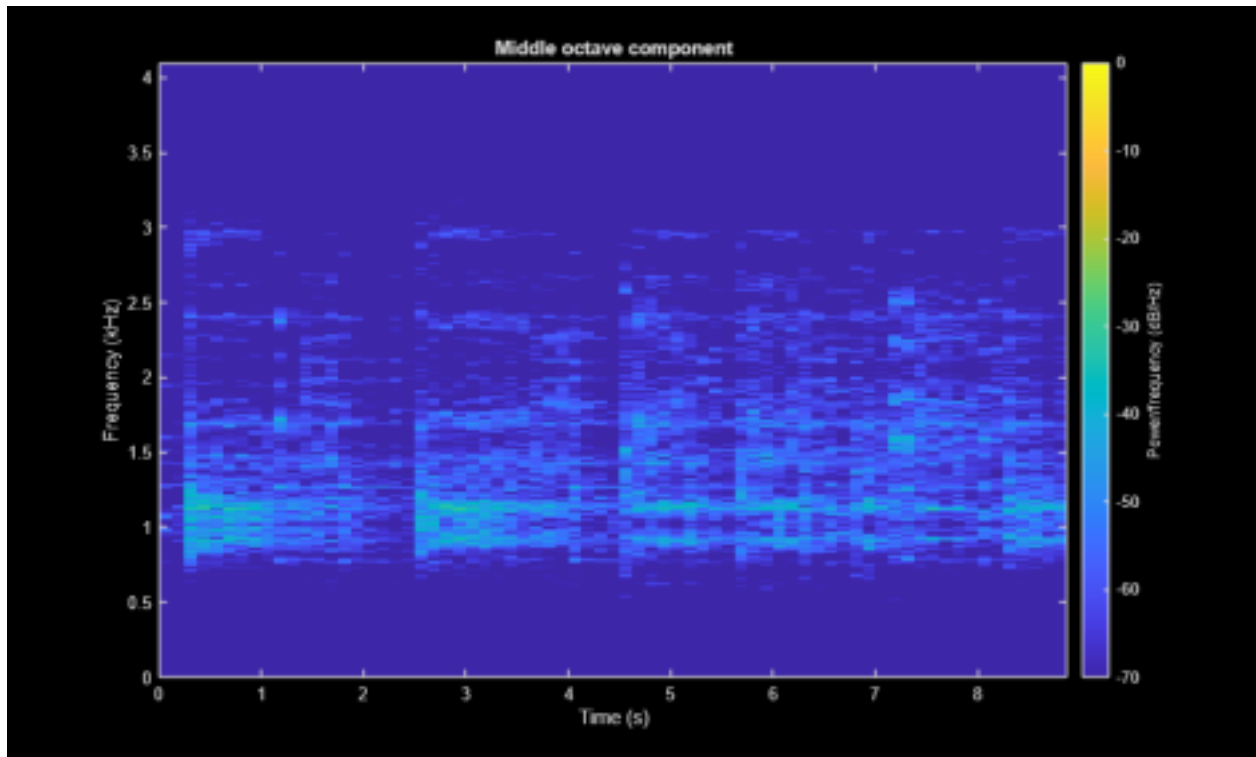
```



High octave component spectrogram.

```
middle_octave = conv(upsample(L1H2,2),HS);
middle_octave = conv(upsample(middle_octave,2),LS);
figure;

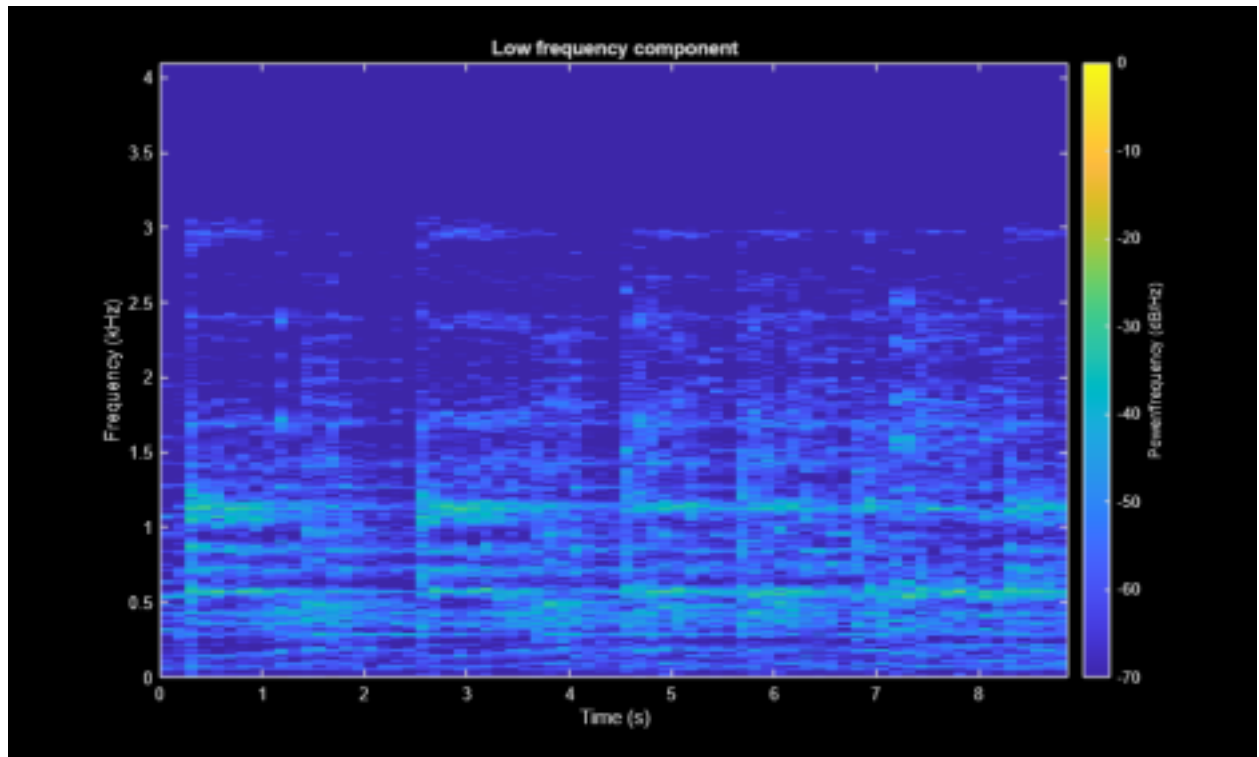
spectrogram(middle_octave,1024,0,1024,'yaxis',fs);
clim([-70,0]); title('Middle octave component')
```



Middle octave spectrogram

```
low_component = conv(upsample(L1,2),LS);
figure;

spectrogram(low_component,1024,0,1024,'yaxis',fs);
clim([-70,0]); title('Low frequency component');
```



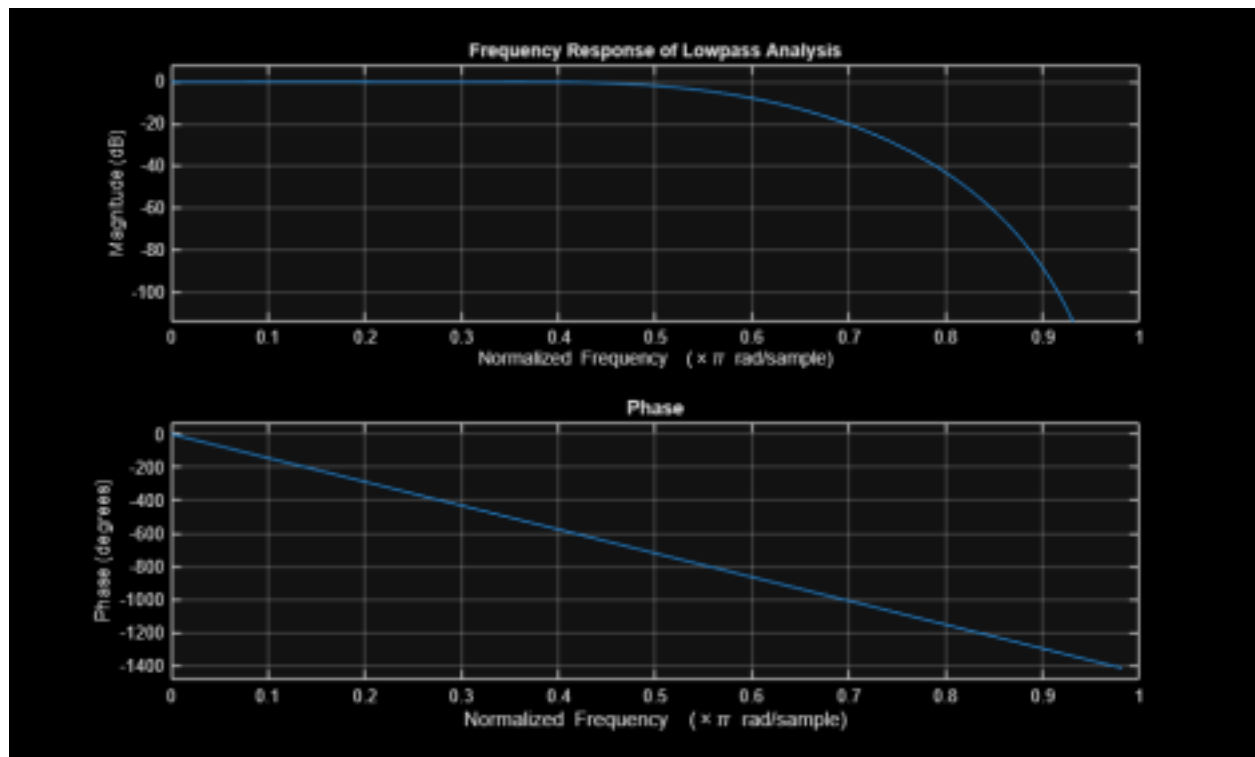
Low frequency spectrogram

3.1 Individual Filter Responses

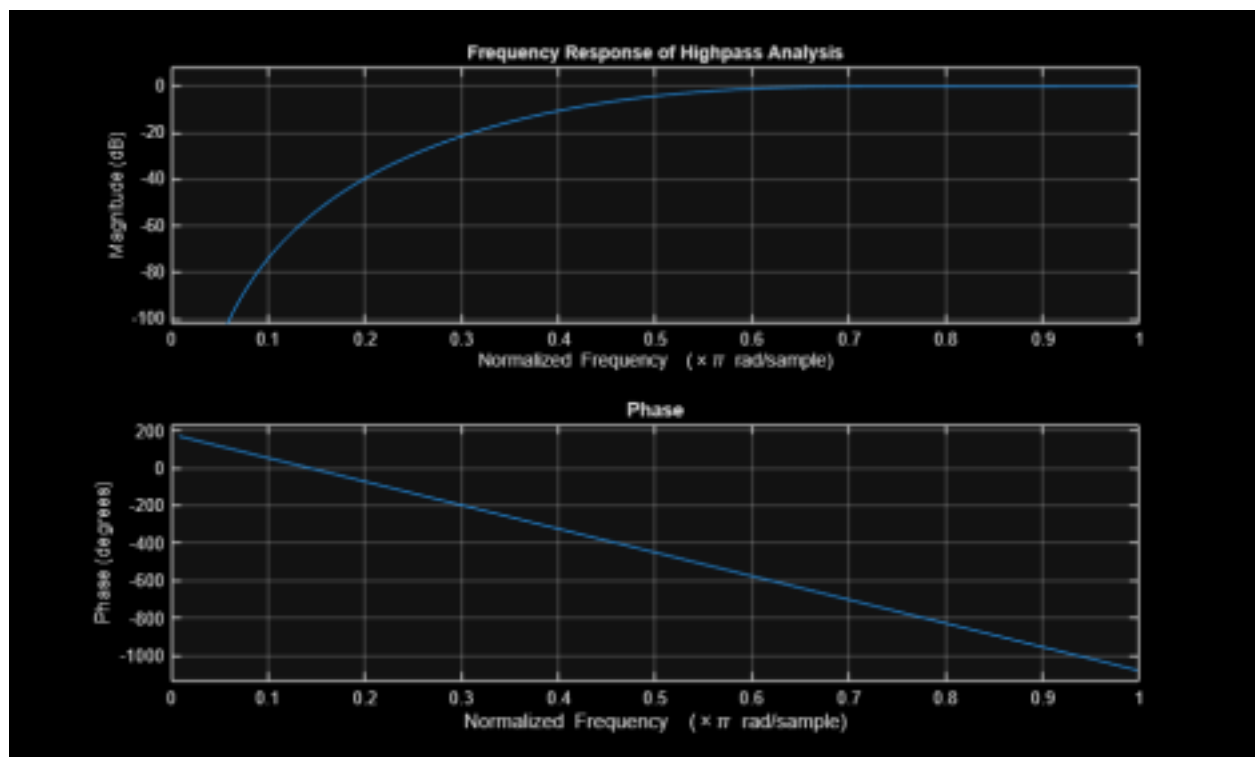
The four FIR filters serve complementary roles in the analysis and synthesis stages of the filter bank. The analysis filters (LA and HA) decompose the input signal into frequency subbands, while the synthesis filters (LS and HS) reconstruct the output from the processed subband signals. We first examine the frequency responses of these individual filters to understand their spectral characteristics.

`%our code`

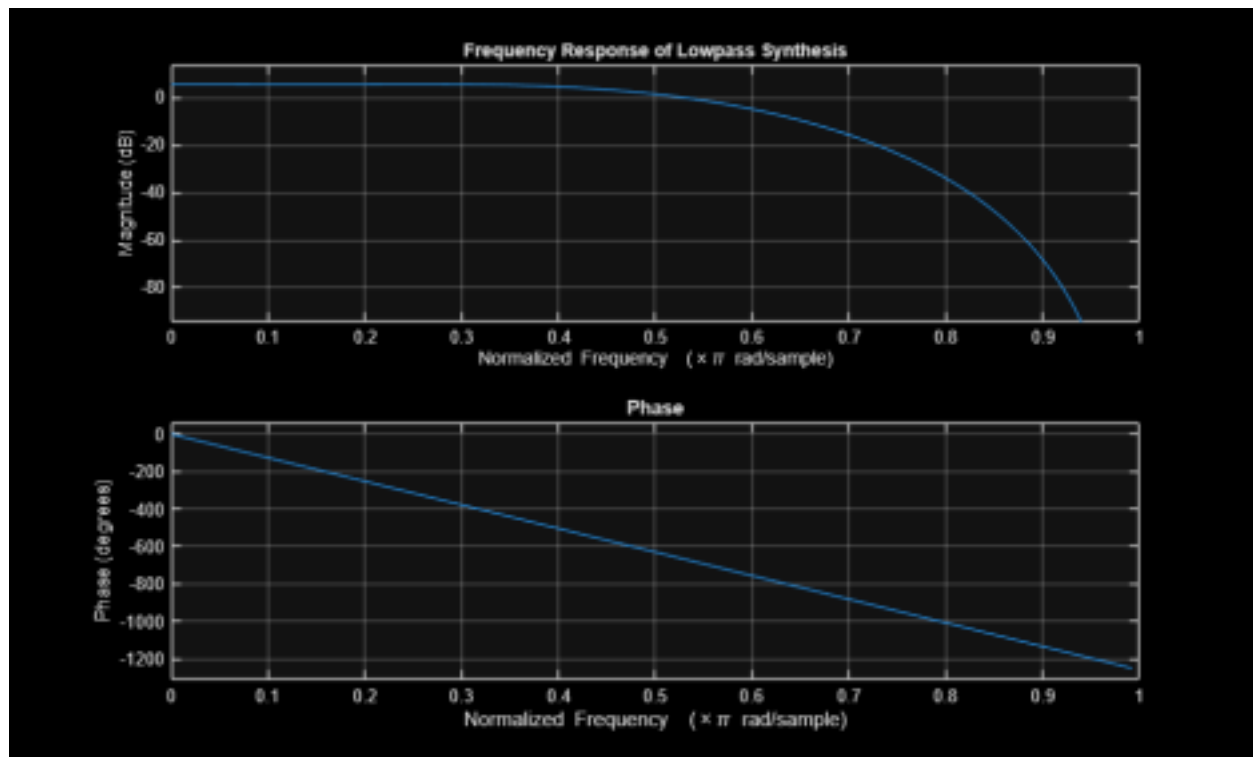
```
%3.1
figure; freqz(LA); title("Frequency Response of Lowpass Analysis");
figure; freqz(HA); title("Frequency Response of Highpass Analysis");
figure; freqz(LS); title("Frequency Response of Lowpass Synthesis");
figure; freqz(HS); title("Frequency Response of Highpass
Synthesis");
```



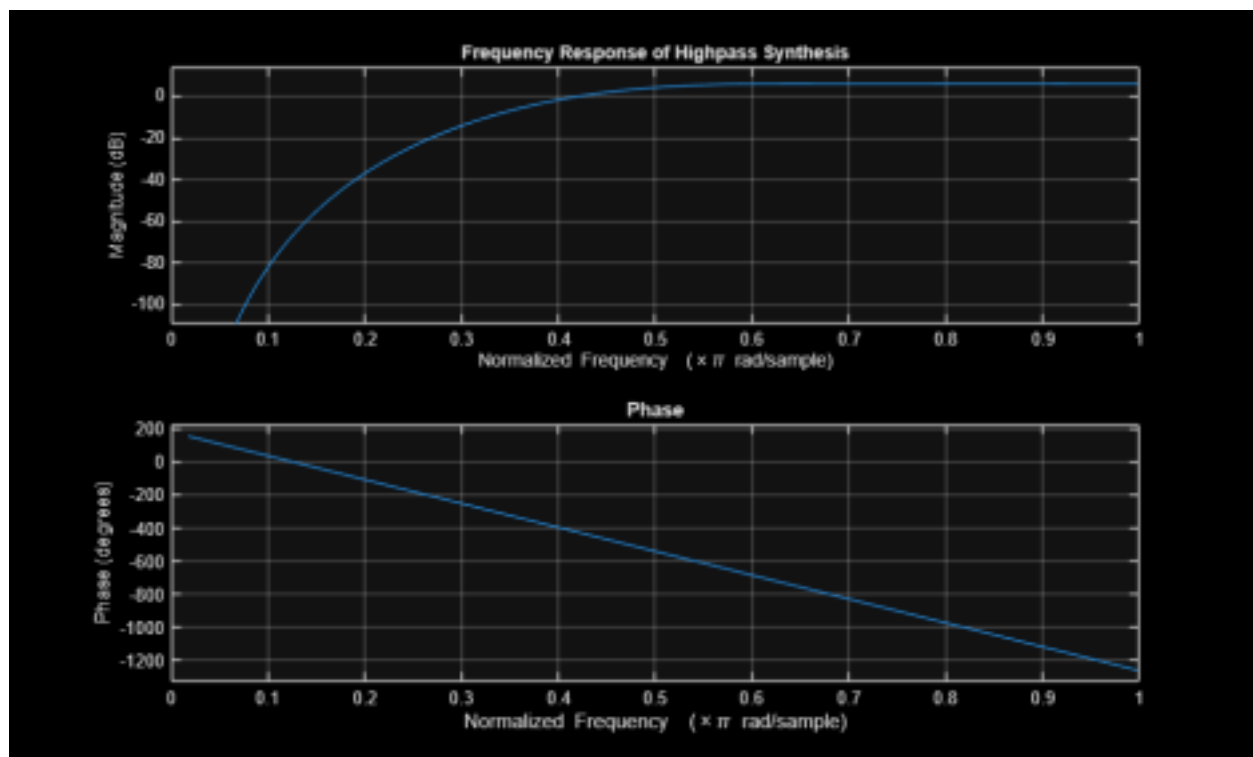
Frequency and phase of the low-pass filter



Frequency and phase of the high-pass filter



Frequency and phase of the low-pass synthesis

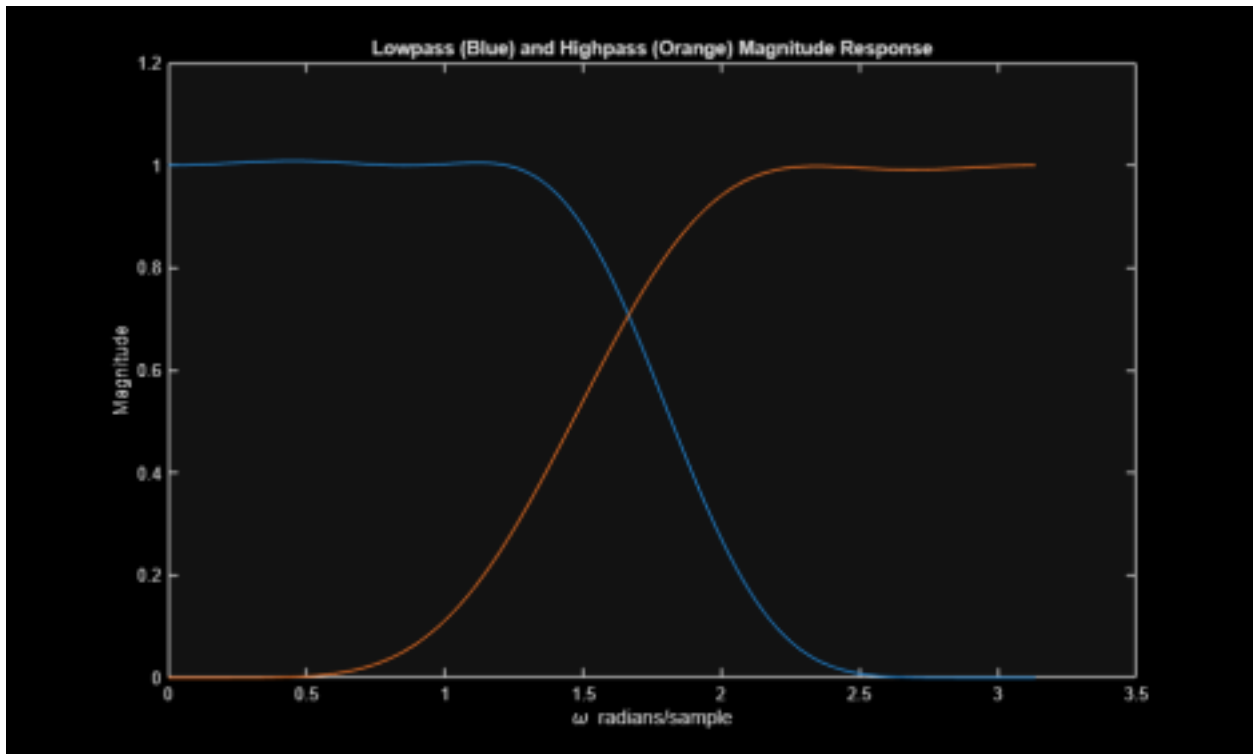


Frequency and phase of the high-pass synthesis

3.2 Plotting the Individual Filter Responses

To understand how the filter bank decomposes the input signal, we plot the magnitude responses of the lowpass analysis (LA) and highpass analysis (HA) filters on the same graph. This visualization shows how these two filters partition the frequency spectrum during the initial analysis stage, with LA passing low frequencies and HA passing high frequencies.

```
%3.2
figure;
[LAH, LAw] = freqz(LA, 1, 512);
plot(LAw, abs(LAH));
hold on;
[HAH, HAw] = freqz(HA, 1, 512);
plot(HAw, abs(HAH));
title("Lowpass (Blue) and Highpass (Orange) Magnitude Response");
xlabel("\omega radians/sample");
ylabel("Magnitude");
```



Low and High Magnitude responses

3.2 Effective Branch Frequency Responses

Using the theoretical results from Section 2.0 (problems 2.3 and 2.4), we calculate the effective frequency response for each of the three complete branches in the filter bank. The multirate operations introduce frequency scaling and gain factors that modify how the individual filter responses combine. We now plot all three magnitude responses on the same graph to visualize how the filter bank partitions the entire frequency spectrum into three distinct octave bands.

```
%3.3
```

```

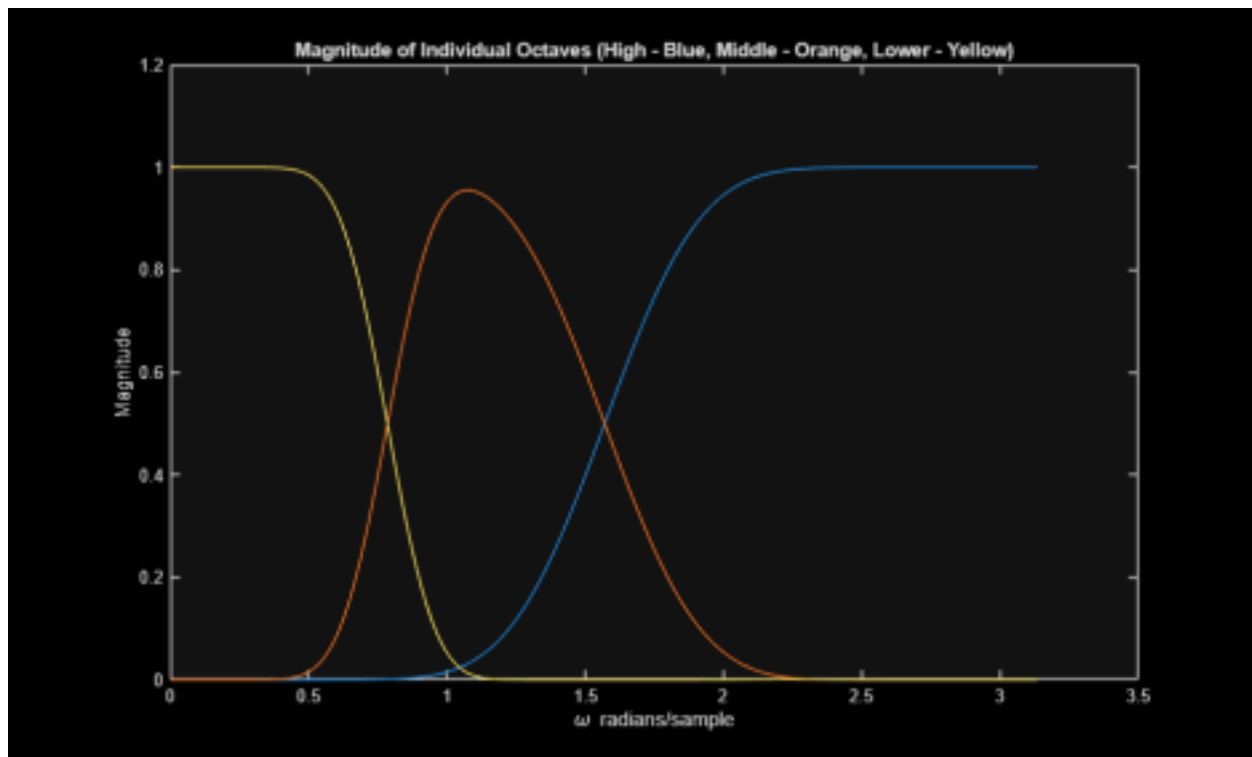
[HSh, HSW] = freqz(HS, 1, 512);
[LSh, LSW] = freqz(LS, 1, 512);

HAH2 = downsample([HAH; flipud(conj(HAH))],2);
HSh2 = downsample([HSh; flipud(conj(HSh))],2);
LAh2 = downsample([LAH; flipud(conj(LAH))],2);
LSh2 = downsample([LSh; flipud(conj(LSh))],2);

highOctaveCoeffs = 0.5 * HAH .* HSh;
middleOctaveCoeffs = 0.25 * LAH .* HAH2 .* HSh2 .* LSh;
lowOctaveCoeffs = 0.25 * LAH .* LAh2 .* LSh2 .* LSh;

figure;
plot(LSW, abs(highOctaveCoeffs));
hold on;
plot(LSW, abs(middleOctaveCoeffs));
hold on;
plot(LSW, abs(lowOctaveCoeffs));
title("Magnitude of Individual Octaves (High - Blue, Middle - Orange, Lower - Yellow)");
xlabel("\omega radians/sample");
ylabel("Magnitude");

```



Magnitude of 3 octave branches

4.0 Extending to more octaves.

We then recursively decomposed the low-frequency branch, which divided the signal into a greater number of octaves. Starting by setting the range so that there are five branches,

then

resampling a recording of a violin into those five branches. Afterwards, we reconstructed each branch into a spectrogram (Low, octave 1, 2, 3, and 4). In a multi-level decomposition, each octave covers a specific frequency range. The highest octave spans from $fs/4$ to $fs/2$, the next octave covers $fs/8$ to $fs/4$, and so on. This logarithmic frequency division matches the structure of musical scales, where each octave represents a doubling of frequency.

4.1 Four-Level Filter Bank Implementation

Here, we extend the two-level filter bank to four levels by applying the decomposition recursively to the low-frequency branch. Using a sampling rate of $fs = 8134$ Hz, the highest octave corresponds to 2033.5 Hz to 4067 Hz (C7 to B7 on the western musical scale). With four levels, middle C (261.62 Hz) is now included in its own octave branch rather than being part of the low-frequency branch.

The four-level structure creates five output branches: four octave-spaced branches and one low-frequency branch covering the remaining frequencies below the lowest octave. %4.1

```
% Analysis (level 1)
%L1 = downsample(conv(x, LA), 2);
%H1 = downsample(conv(x, HA), 2);

% Analysis (level 2)
L2 = downsample(conv(L1, LA), 2);
H2 = downsample(conv(L1, HA), 2);

% Analysis (level 3)
L3 = downsample(conv(L2, LA), 2);
H3 = downsample(conv(L2, HA), 2);

% Analysis (level 4)
L4 = downsample(conv(L3, LA), 2);
H4 = downsample(conv(L3, HA), 2);
```

4.2 Violin Audio File Analysis

To test the extended filter bank, we resample the violin-C4 recording to 8134 Hz and process it through the four-level system. We verify that the synthesis transform successfully reconstructs the original signal by comparing the output to the input, ensuring the filter bank achieves near-perfect reconstruction.

%4.2

Resampled

```
% Audio resample
violinWav = 'violin-C4.wav';
[violin, fs] = audioread(violinWav);
violinSound = resample(violin, 8134, fs);
```

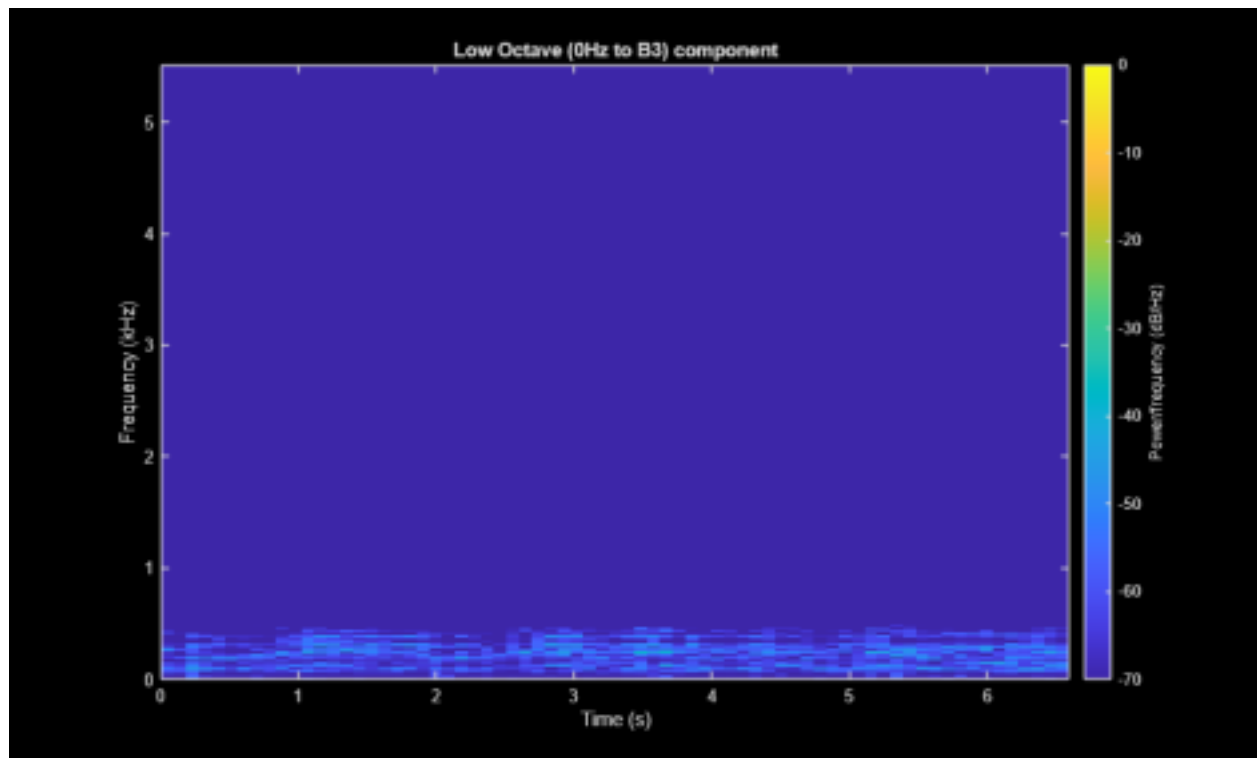
4.3 Individual Branch Reconstruction and Spectrograms

Finally, we reconstruct each of the five branches independently to analyze their frequency content over time. For each branch, we create a spectrogram showing how the energy in that

octave varies throughout the recording. These spectrograms reveal how the violin's energy is distributed across different octaves and how it evolves over the duration of the note.

%4.3

```
lowerOctave = conv(upsample(L4, 2), LS);  
lowerOctave = conv(upsample(lowerOctave, 2), LS);  
lowerOctave = conv(upsample(lowerOctave, 2), LS);  
lowerOctave = conv(upsample(lowerOctave, 2), LS); figure;  
spectrogram(lowerOctave,1024,0,1024,'yaxis',fs);  
clim([-70,0]); title('Low Octave (0Hz to B3) component');
```



```
octaveOne = conv(upsample(H4, 2), HS);  
octaveOne = conv(upsample(octaveOne, 2), LS); octaveOne  
= conv(upsample(octaveOne, 2), LS); octaveOne =  
conv(upsample(octaveOne, 2), LS); figure;  
spectrogram(octaveOne,1024,0,1024,'yaxis',fs);  
clim([-70,0]); title('Octave One (C4 to B4) component');
```



```
octaveTwo = conv(upsample(H3, 2), HS);  
octaveTwo = conv(upsample(octaveTwo, 2), LS); octaveTwo =  
conv(upsample(octaveTwo, 2), LS); figure;  
spectrogram(octaveTwo,1024,0,1024,'yaxis',fs);  
clim([-70,0]); title('Octave Two (C5 to B5) component');
```



```
octaveThree = conv(upsample(H2, 2), HS);  
octaveThree = conv(upsample(octaveThree, 2), LS); figure;  
spectrogram(octaveThree,1024,0,1024,'yaxis',fs);  
clim([-70,0]); title('Octave Three (C6 to B6) component');
```



```
octaveFour = conv(upsample(H1, 2), HS);  
figure; spectrogram(octaveFour,1024,0,1024,'yaxis',fs);  
clim([-70,0]); title('Octave Four (C7 to B7) component');  
% octave 4 is the highest and 1 is the lowest
```



5.0 Playback and use as compression system

The octave-band filter bank can be used as a compression system by taking note of the fact that lower frequency bands require fewer samples after downsampling. In this section, we analyze the compression performance of the four-level filter bank by processing an audio recording and examining each band independently. We evaluate the trade-off between compression ratio and signal quality to determine which band provides the most efficient representation. %5.0

```
wabashWav = 'wabashCannonball.wav';  
[wabash, fs1] = audioread(wabashWav);  
% resample to 8134Hz  
wabashSong = resample(wabash, 8134, fs1);  
% truncate to 25 seconds  
wabashSong = wabashSong(8134:138277);
```

5.1 Audio Signal Preparation and Band Decomposition

We load an audio recording of "Wabash Cannonball" and prepare it for analysis. The signal is resampled to 8134 Hz and converted to mono by averaging the stereo channels. We then truncate the signal to exactly 130144 samples (16 seconds) to match the filter bank processing requirements.

Using the four-level filter bank from Section 4.0, we decompose the signal into five bands: four octave branches and one low-frequency branch. Each level of downsampling reduces the number of samples by a factor of two, creating different compression ratios for each band.

```
%5.1
% original has 130144 samples
% Analysis (level 1)
WabashL1 = downsample(conv(wabashSong, LA), 2);
WabashH1 = downsample(conv(wabashSong, HA), 2);
% 65072 samples
% CR1 = 65072/130144 = 1/2
```

4

```
% Analysis (level 2)
WabashL2 = downsample(conv(WabashL1, LA), 2);
WabashH2 = downsample(conv(WabashL1, HA), 2);
% 32536 samples
% CR2 = 32536/130144 = 1/4
```

```
% Analysis (level 3)
WabashL3 = downsample(conv(WabashL2, LA), 2);
WabashH3 = downsample(conv(WabashL2, HA), 2);
% 16268 samples
% CR3 = 16268/130144 = 1/8
```

```
% Analysis (level 4)
WabashL4 = downsample(conv(WabashL3, LA), 2);
WabashH4 = downsample(conv(WabashL3, HA), 2);
% 8134 samples
% CR4 = 8134/130144 = 1/16
```

```
sampleFreq = 8134;
```

5.2 Compression Ratio Analysis

After the analysis transform and prior to upsampling, each band requires a different number of samples. The compression ratio represents the fraction of samples needed to store each band compared to the original signal, assuming the same precision is used for all samples. Lower frequency bands achieve higher compression ratios because they are downsampled more times.

```
%5.2
```

```
WabashLowerOctave = conv(upsample(WabashL4, 2), LS);
WabashLowerOctave = conv(upsample(WabashLowerOctave, 2), LS);
WabashLowerOctave = conv(upsample(WabashLowerOctave, 2), LS);
WabashLowerOctave = conv(upsample(WabashLowerOctave, 2), LS);
```

```
WabashOctaveOne = conv(upsample(WabashH4, 2), HS);
WabashOctaveOne = conv(upsample(WabashOctaveOne, 2), LS);
WabashOctaveOne = conv(upsample(WabashOctaveOne, 2), LS);
WabashOctaveOne = conv(upsample(WabashOctaveOne, 2), LS);
WabashOctaveTwo = conv(upsample(WabashH3, 2), HS);
WabashOctaveTwo = conv(upsample(WabashOctaveTwo, 2), LS);
```

```

WabashOctaveTwo = conv(upsample(WabashOctaveTwo, 2), LS);

WabashOctaveThree = conv(upsample(WabashH2, 2), HS);
WabashOctaveThree = conv(upsample(WabashOctaveThree, 2), LS);

WabashOctaveFour = conv(upsample(WabashH1, 2), HS);

```

5.3 Individual Band Playback

We reconstruct each of the five bands independently and play them back to understand their perceptual characteristics, our observations of which can be seen in the MATLAB comments below.

```

%5.3
%{
soundsc(wabashSong); % original sound pause(18);
soundsc(WabashLowerOctave, sampleFreq); % lower octave
pause(18);
% In this octave I can only really hear the bass drum and noise produced in
% the lower octave. It sounds like the band is playing the song in a %
different room and I can barely hear it through the walls.

```

5

```

soundsc(WabashOctaveOne, sampleFreq); % octave one
pause(18);
% I can mainly hear the percussion parts and some of the tuba range in this
% octave. It kind of sounds like the band is underwater and the sound is %
muddled.
soundsc(WabashOctaveTwo, sampleFreq); % octave two
pause(18);
% This octave sounds very noisy and the sound has a lot of apparent digital
% sound. It sounds a little similar to the original recording but with a %
lot of processing done to make the signal noisy.
soundsc(WabashOctaveThree, sampleFreq); % octave three
pause(18);
% Describe
soundsc(WabashOctaveFour, sampleFreq); % octave four
% This octave just sounds like a low quality recording with no bass parts.
% The
%}

```

5.4 PSNR and Compression Trade-off Analysis

In order to assess signal quality, we compute the Peak Signal-to-Noise Ratio (PSNR) for each reconstructed band compared to the original signal:

- Low frequency branch: PSNR = 18.24 dB, CR = 1/16
- Octave 1: PSNR = 18.79 dB, CR = 1/16
- Octave 2: PSNR = 18.52 dB, CR = 1/8
- Octave 3: PSNR = 18.36 dB, CR = 1/4
- Octave 4: PSNR = 18.66 dB, CR = 1/2

Plotting PSNR versus compression ratio reveals that **Octave 1 provides the best trade-off**. It achieves the highest PSNR (18.79 dB) while maintaining the maximum compression ratio (1/16). Although Octave 4 has a comparable PSNR, it requires twice as many samples for storage, making Octave 1 the most efficient choice for this signal.

The full reconstruction, obtained by summing all five bands, closely approximates the original signal, demonstrating that the filter bank achieves near-perfect reconstruction when all bands are combined.

%5.4

```
PSNRlow = -10*log10(mean(abs(wabashSong -
WabashLowerOctave(1:130144)).^2)) + 6.02;
PSNRone = -10*log10(mean(abs(wabashSong - WabashOctaveOne(1:130144)).^2)) +
6.02;
PSNRtwo = -10*log10(mean(abs(wabashSong - WabashOctaveTwo(1:130144)).^2)) +
6.02;
PSNRthree = -10*log10(mean(abs(wabashSong -
WabashOctaveThree(1:130144)).^2)) + 6.02;
PSNRfour = -10*log10(mean(abs(wabashSong - WabashOctaveFour(1:130144)).^2)) +
6.02;
```

```
%{
PSNRlow = 18.2434 CR = 1/16
PSNRone = 18.7944 CR = 1/16
PSNRtwo = 18.5173 CR = 1/8
PSNRthree = 18.3631 CR = 1/4
PSNRfour = 18.6628 CR = 1/2
```

It looks like the best trade-off between PSNR and Compression Rate happens in Octave One. The PSNR is the best over all of the other octaves, though not much better than octave four, and the compression is the highest, tied with lower octave.

```
%}
```